

RELATÓRIO TÉCNICO
TRABALHO PRÁTICO PARTE 1: COMUNICAÇÃO CONFIÁVEL
INTER-PROCESSOS

Integrantes:
Bernardo Callegari Boeira
Eduardo Camozzato Fonte
Paulo Henrique Ceccato

A. TRABALHO PROPOSTO

Este relatório visa apresentar a implementação de um serviço distribuído seguindo a arquitetura cliente-servidor que realiza a soma de números inteiros. Implementado com linguagem Python e utilizando exclusivamente a API User Datagram Protocol, o sistema tem como objetivo receber, de múltiplos clientes, requisições contendo valores numéricos inteiros e adicioná-los a uma variável acumuladora central mantida pelo servidor.

B. IMPLEMENTAÇÃO

a. Descoberta

Este subserviço foi implementado primariamente no módulo `services/discovery.py`. O cliente, ao iniciar, utiliza um socket UDP configurado para aceitar broadcast (`SO_BROADCAST`). Ele detecta dinamicamente o endereço de broadcast da rede local (usando `ioctl` no Linux ou por inferência de sub-rede no Windows) e envia mensagens do tipo `DISCOVERY` para a porta especificada. O servidor, que está escutando na mesma porta, recebe essa mensagem e responde com uma mensagem `DISCOVERY_RESPONSE` contendo o seu próprio IP e porta diretamente para o IP do cliente (unicast). A escolha de broadcast foi feita para permitir que os clientes encontrem o servidor sem a necessidade de configurações de IPs prévios, tornando o sistema escalável.

b. Processamento

Responsável por garantir a entrega *exactly-once* e implementar a lógica de negócio. No cliente (`services/processing.py` via `cliente.py`), o processamento atua ativamente, consumindo de uma fila interna os valores a serem enviados. Ele anexa a cada mensagem um `id_req` sequencial, iniciando em 1. Após enviar a requisição, o cliente entra em um loop onde aguarda um `ACK`. Caso ocorra um timeout (definido como 10ms), ele retransmite a mensagem. No servidor (`services/processing.py`), o processamento é passivo e regido por regras de estado para cada cliente. O servidor avalia o `id_req` recebido contra o `last_req` armazenado. Se `id_req == last_req + 1`, a variável global é incrementada. Caso contrário (mensagem duplicada ou fora de ordem), o servidor omite a soma global e responde com o último `ACK` armazenado.

c. Interface

Implementado nos pontos de entrada de ambos os processos (`servidor.py` e `cliente.py`). No servidor, os prints com timestamp e o estado atual das somas (e ocorrências de pacotes DUP!!) foram colocados de forma intercalada aos métodos de processamento das mensagens. No cliente, o subserviço de interface foi implementado usando a abordagem multi-thread. Uma thread dedicada (`reader_thread`) é exclusiva para ler a entrada padrão do usuário e repassar os valores via fila (`queue.Queue`) para a thread de

processamento/envio (`sender_thread`), enquanto os prints de resposta são feitos na tela de forma assíncrona.

C. SINCRONIZAÇÃO NO ACESSO A DADOS

Foi mandatório aplicar sincronização no servidor, pois ele lida com requisições simultâneas vindas de múltiplos clientes através do protocolo UDP. Embora o event loop do nosso socket seja de certa forma sequencial devido ao `recvfrom`, o processamento e a manipulação do estado requerem atomicidade.

A sincronização ocorre no acesso às variáveis que definem o estado global do servidor. No arquivo `models/server_state.py`, instanciou-se um objeto `threading.Lock()`. No arquivo `services/processing.py`, na função `server_handle_request`, toda a lógica de negócio (validar o `id_req` de um cliente específico e atualizar as variáveis `num_reqs` e `total_sum`) foi colocada dentro de um bloco `with state.lock:`. Isso garante que se duas mensagens chegarem em rápida sucessão, o estado da soma não será corrompido devido a *race conditions* (condições de corrida), garantindo consistência no armazenamento.

D. PRINCIPAIS ESTRUTURAS E FUNÇÕES

As principais estruturas desenvolvidas visam guardar o estado da comunicação para atender às exigências de *exactly-once* sem estourar o limite de memória ou variáveis:

- **ServerState (em `models/server_state.py`):** Mantém as variáveis globais (`num_reqs` e `total_sum`) e um dicionário (`clients`), onde a chave é o IP/porta do cliente. A função `add_value` aplica uma máscara bitwise (`_UINT64_MASK = (1 << 64) - 1`) na soma total, para garantir estritamente o comportamento equivalente a um tipo numérico `unsigned int` de 64 bits, prevenindo overflows no Python, que normalmente possui inteiros de tamanho arbitrário.

- **ClientState (em `models/client_state.py`):** Mantém a informação "por cliente" no servidor. Guarda qual foi o `last_req` recebido e validado, o `last_num_reqs` e o `last_total_sum` no instante em que a requisição daquele cliente foi validada. Estes últimos são cruciais para que, caso um pacote de ACK se perca na rede, o servidor consiga reenviar a exata resposta passada sem precisar somar o número duas vezes.

- **protocol.py e funções de serialização (encode/decode):** Toda mensagem transmitida no sistema obedece a um protocolo simples em JSON (em UTF-8). Funções ajudantes como `make_request`, `make_ack` abstraem a complexidade da rede e convertem os parâmetros de domínio para objetos prontos para trânsito via socket UDP.

E. PRIMITIVAS DE COMUNICAÇÃO INTER-PROCESSOS USADAS

Para a comunicação, escolheu-se utilizar estritamente a API de Sockets nativa da linguagem (biblioteca padrão `socket`), sem bibliotecas de terceiros, operando sobre o protocolo de transporte UDP (User Datagram Protocol). As primitivas instanciadas (disponíveis no módulo `network/udp.py`) foram:

- **`socket.socket(socket.AF_INET, socket.SOCK_DGRAM)`:** Inicializa a comunicação inter-processos sobre IPv4 em formato de datagramas (UDP).

- **`setsockopt(socket.SOL_SOCKET, socket.SO_BROADCAST, 1)`:** Usada explicitamente para habilitar o envio e recebimento de pacotes que têm como destino toda a rede (essencial para a fase de descoberta).

- **`bind()`:** Associa o socket à porta parametrizada. No cliente, vinculou-se à porta 0, deixando a cargo do SO a escolha de uma porta aleatória disponível para envio das requisições.

- **`sendto()` e `recvfrom()`:** Respectivamente usadas para enviar datagramas e ler datagramas vindos da rede. Por ser UDP, não há conexão persistente (como no TCP), então essas primitivas são invocadas informando ou retornando explicitamente uma tupla contendo (Endereço IP, Porta) da outra ponta a cada transação, permitindo o servidor discriminar diferentes clientes de forma imediata (identificando-os pela tupla recebida por `recvfrom()`).

F. PROBLEMAS ENFRENTADOS E SUAS SOLUÇÕES

a. Endereçamento de Broadcast Fixo

Inicialmente o código estava fixando o endereço 255.255.255.255, que é o endereço de broadcast local. Isto estava funcionando nos testes executados na mesma máquina, porém ao testar em máquinas separadas, pode ocorrer o problema do cliente não encontrar o servidor na fase de descoberta. Para a solução desse

problema foi implementada a função **get_broadcast_addr()**, que detecta automaticamente o broadcast correto da interface de rede ativa da máquina.

- b. Eventos do Socket UDP Descartando Requisições (Sincronização no Cliente):

Havia o problema de se ter o cliente tentando enviar dados sequenciais e ficar preso aguardando ACKs perdidos. Resolução: Foi imperativo a divisão do cliente em múltiplas threads usando `threading.Thread` e `queue.Queue`. Enquanto a thread principal (reader) coleta tudo que o usuário envia via entrada padrão rapidamente e joga numa fila atômica assíncrona, a thread de envio (sender) consome uma por vez, disparando timeouts de 10ms até que as garantias de confirmação sejam provadas via ACKs UDP. Assim, o sistema atende aos testes de robustez e duplicação garantindo processamento.

- c. Quantidade de mensagens com Ack duplicado no wi-fi:

Em virtude de estabelecer uma conexão via wi-fi para a troca de mensagens, a latência na troca de pacotes, muitas vezes era maior que 10 milissegundos, fazendo com que fossem enviadas duplicatas. Diante disso, a solução passou por aumentar o valor de timeout, para que não fossem enviadas tantas duplicatas e fosse mais rápido a troca de mensagens.